

Manual 02 - Autenticacion y Sesion

Version: 1.1

1) Explicacion no tecnica

Que hace esta parte

Resuelve 3 preguntas clave:

- Quien puede entrar.
- Cuanto tiempo permanece adentro.
- Que pasa cuando el acceso deja de ser valido.

Flujo simple para cualquier persona

1. El usuario escribe correo y clave.
2. El servidor responde si las credenciales son validas.
3. Si son validas, la app guarda una "llave temporal" (token).
4. Con esa llave se habilita la navegacion interna.
5. Cuando la llave vence, la app cierra sesion automaticamente.

Ejemplo no tecnico

Escenario:

- Sofia inicia sesion a las 09:00.
- A las 12:00 su token ya vencio.
- Al abrir una vista, la app detecta token vencido y la envia a Login.

Resultado:

- No queda sesion "fantasma" abierta.
- Se evita que alguien use una sesion vieja.

2) Explicacion tecnica

Archivos clave

- src/components/login/Login.jsx
- src/service/localStorage.js
- src/service/api.js
- src/App.jsx

Flujo tecnico de Login

1. Login.jsx hace POST a /api/v1/auth/login.
2. Espera respuesta con formato: { token, user }.
3. setAuthToken(token) configura Authorization para llamadas futuras.

4. setUserSession(user) normaliza campos y guarda currentUser.
5. Se dispara evento authChanged para refrescar componentes.
6. navigate() redirige a la ruta de destino.

Ejemplo tecnico de request/response

Request:

```
{
  "email": "trabajador@roomops.com",
  "password": "123456"
}
```

Response esperada:

```
{
  "token": "eyJhbGciOi...",
  "user": {
    "id": 17,
    "email": "trabajador@roomops.com",
    "firstName": "Sofia",
    "lastName": "Diaz",
    "role": "TRABAJADOR",
    "activo": true
  }
}
```

Donde se guarda la sesion

En localStorage:

- token
- isLoggedIn
- currentUser

currentUser queda normalizado para evitar diferencias entre frontend y backend.

Validacion de expiracion

isTokenExpired(token):

- decodifica payload JWT
- lee claim exp
- compara con Date.now()

isUserLoggedIn():

- valida flag de sesion

- valida token
- si no cumple, ejecuta clearUserSession()

Fallback de robustez

Si user llega incompleto en Login.jsx:

- se decodifican claims del token
- se arma sesion minima con id/email/role

Esto evita que un 403 posterior en otro endpoint rompa la sesion inicial.

3) Flujo completo con ejemplos

Caso A - Login exitoso normal

1. Usuario envia credenciales validas.
2. API devuelve token + user.
3. App guarda sesion.
4. Sidebar muestra modulos por rol.

Caso B - Token vencido

1. Usuario mantiene pestaña abierta.
2. Token vence por tiempo.
3. En siguiente chequeo, isUserLoggedIn devuelve false.
4. App limpia sesion y redirige a /login.

Caso C - Role con prefijo

Si backend envia ROLE_SUPERVISOR:

- getUserRole normaliza a SUPERVISOR.

Resultado:

- No falla la matriz de permisos por formato de string.

4) Problemas frecuentes y solucion

- Problema: usuario entra y no ve modulos
 - Revisar campo role en currentUser.
 - Revisar normalizacion en getUserRole.
- Problema: cierre de sesion inesperado
 - Verificar exp del token.
 - Verificar hora del sistema cliente.
- Problema: backend retorna 403 en consulta de usuario
 - Mantener sesion con datos de login.

- Evitar depender de endpoint no permitido para completar sesion.

5) Buenas practicas de mantenimiento

1. Mantener contrato de login estable: { token, user }.
2. Normalizar rol siempre en un solo lugar.
3. No duplicar logica de expiracion en muchos componentes.
4. Al cambiar auth backend, actualizar este manual y tests.